

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

CI-0117 Programación Paralela y Concurrente

Reporte:

Simulación de la Ecuación de Calor 3D mediante Computación Paralela (OpenMP)

Profesor:

Josef Ruzicka González.

Estudiantes:

Jimena Avella Chavarria C4C850

Luis Davis Carvajal Villalobos (C4D828)

I Semestre, 2026

Sede Rodrigo Facio

Reporte

Para la implementación secuencial, se empleó un bloque de memoria dinámica, representando la malla 3D de tamaño $N \times N \times N$. Se generaron dos arreglos idénticos, *cube_old* y *cube_new*, donde *cube_old* actúa como estado de solo lectura durante cada iteración, y *cube_new* almacena los cálculos actualizados del *estencil*. Las condiciones de frontera fueron configuradas de acuerdo con los requisitos del documento: se establecieron en 100 las temperaturas de las caras superior, inferior y las caras laterales, mientras que la cara posterior se fijó en 0. Con el fin de acelerar la convergencia iterativa, todos los puntos internos del cubo se inicializaron con el promedio ponderado de las temperaturas de frontera, logrando así un punto de partida numéricamente más cercano a la solución real en estado estacionario.

La proyección visual del sistema y su validación física se gestionan a través de la función de volcado de datos para ParaView, la cual funciona como un reporte del comportamiento del calor, traduciendo la realidad numérica en archivos con extensión .vtk.

Para evitar que las salidas de datos se reemplacen entre sí, el programa gestiona el nombre de los archivos utilizando una cadena de texto predeterminada a la que se le añade una variable que cuenta e incrementa llamada *numArchive*; de este modo, cada ciclo de guardado genera crónicas independientes. Al abrir el flujo de escritura, el código redacta de forma estricta un encabezado formateado que detalla las propiedades esenciales que la herramienta externa requiere para decodificar el volumen de formato ASCII. Los puntos estructurados, las dimensiones exactas de la malla y el espaciado geométrico, terminando con la declaración de la cantidad total de puntos del dominio, *NTOTAL*, tratados como escalares de doble precisión.

Es importante reconocer que este proceso de exportación de datos impone restricciones físicas y de hardware que causan un desafío en relación a la agilidad del cálculo paralelo, ya que, a que la escritura en el almacenamiento secundario es una operación lenta y secuencial, esta rutina no se puede paralelizar y debe ser ejecutada obligatoriamente por un único hilo para evitar la corrupción del archivo. Además, exige una configuración específica en el orden de los ciclos donde el bucle de la variable K debe posicionarse en el nivel más externo, seguido por J en la sección intermedia, e I en el ciclo más interno. Este ordenamiento inverso es el código exacto que solicita la estructura de puntos de ParaView para reconstruir correctamente el cubo tridimensional.

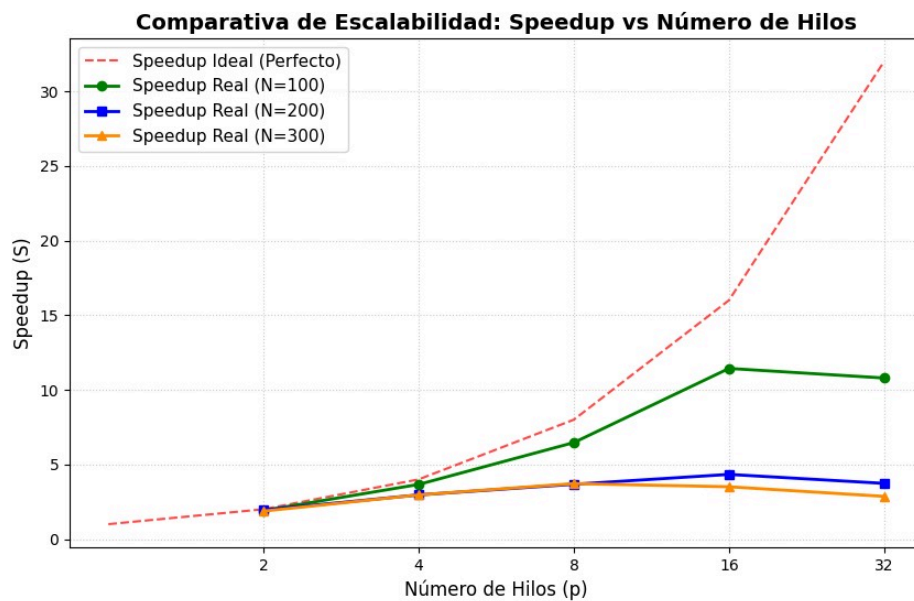
En la ejecución secuencial, el programa recorre todo el espacio interno de la malla realizando el cálculo del *estencil* en cada punto. Una optimización aplicada para reducir el tiempo de cómputo fue evitar la copia masiva de datos desde *cube_new* hacia *cube_old* en cada ciclo; en su lugar, el código implementa un intercambio de punteros mediante la función *std::swap*. Esto convierte un proceso de transferencia de datos de alto costo en una actualización de referencias en memoria de orden constante, mejorando el rendimiento general del algoritmo secuencial.

La estrategia de paralelización se enfocó en el *for* espacial más externo, es decir, el eje correspondiente a las caras del cubo. La directiva de paralelización *#pragma omp parallel for* fue insertado antes del ciclo de iteración principal. Cada hilo lee información únicamente del arreglo *cube_old* y almacena el resultado en una región disjunta de memoria dentro de *cube_new*, eliminando condiciones de carrera.

Para evaluar la eficiencia de esta estrategia, se implementó un estudio de escalamiento variando el tamaño de la malla desde un tamaño base de N=100 hasta N=300, midiendo los tiempos de ejecución mediante la función *omp_get_wtime()*. Los resultados recolectados durante estas pruebas permiten una comparación entre el rendimiento del modelo secuencial y su versión paralela:

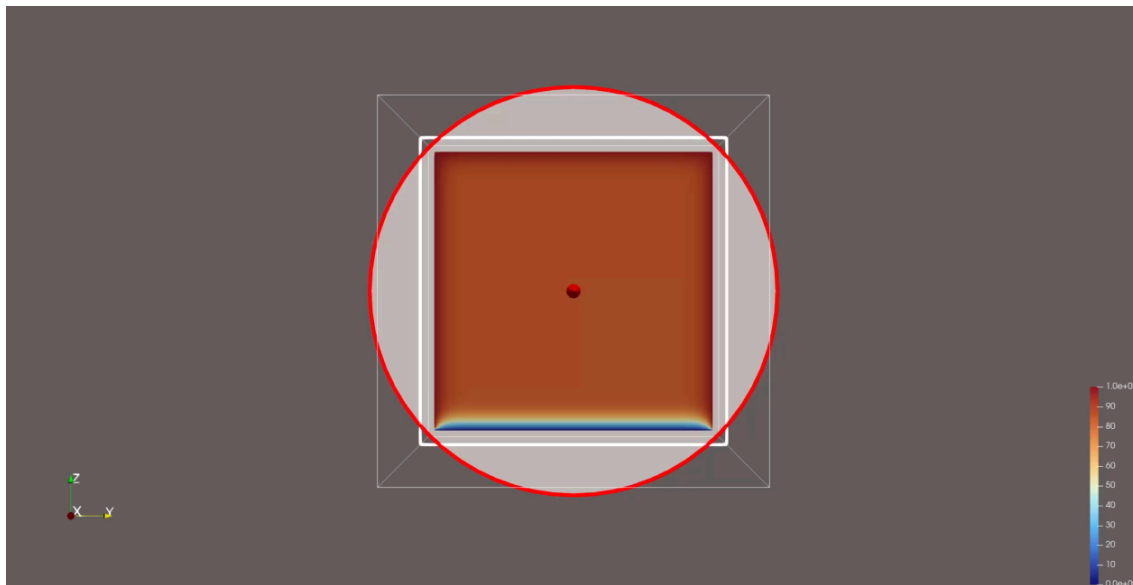
TABLA COMPARATIVA DE RENDIMIENTO (SPEEDUP Y EFICIENCIA)				
Hilos (p)	Métrica	N = 100	N = 200	N = 300
2	Tiempo (s)	0.3917	5.7635	20.5323
	Speedup	1.91	1.94	1.88
	Eficiencia	95.7%	97.2%	93.8%
4	Tiempo (s)	0.2048	3.7836	13.0071
	Speedup	3.66	2.96	2.96
	Eficiencia	91.6%	74.0%	74.0%
8	Tiempo (s)	0.1160	3.0494	10.3575
	Speedup	6.47	3.67	3.72
	Eficiencia	80.8%	45.9%	46.5%
16	Tiempo (s)	0.0656	2.5847	10.9845
	Speedup	11.43	4.33	3.50
	Eficiencia	71.5%	27.1%	21.9%
32	Tiempo (s)	0.0695	3.0008	13.4586
	Speedup	10.79	3.73	2.86
	Eficiencia	33.7%	11.7%	8.9%

Los datos de la tabla de rendimiento demuestran una correlación directa entre el incremento en el volumen de cálculo y la efectividad de la paralelización. En dominios de malla reducida, la sobrecarga asociada a la creación y gestión de los hilos de ejecución de OpenMP atenúa parcialmente las ventajas del paralelismo. Sin embargo, a medida que la resolución de la malla se incrementa hacia dimensiones masivas (como $N=200$ y $N=300$), la carga de trabajo computacional domina el tiempo total de ejecución, revelando reducciones de tiempo drásticas.

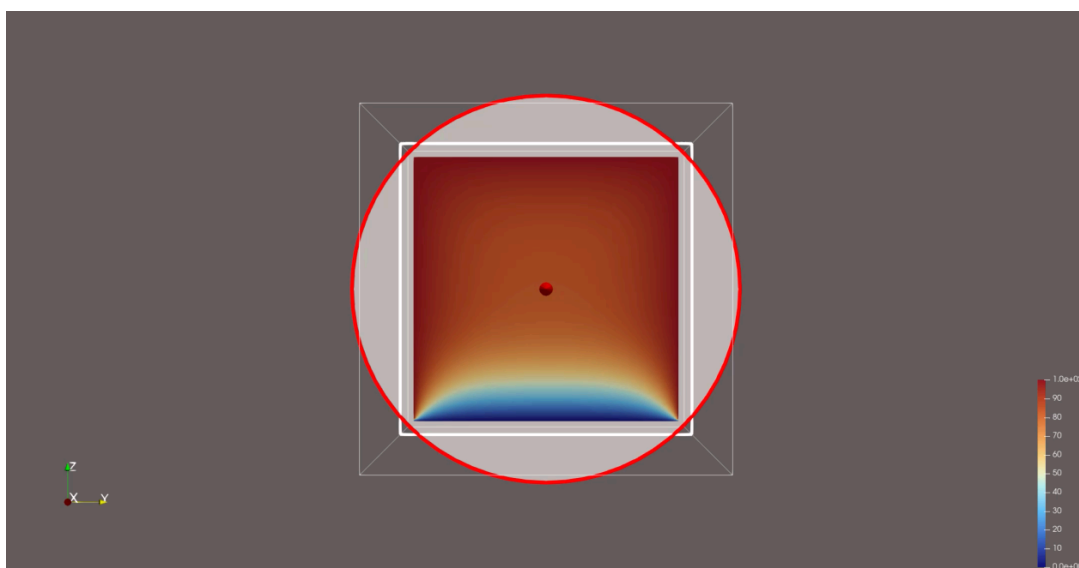


La gráfica confirma que el algoritmo escala de forma casi lineal en los primeros niveles de paralelismo, demostrando una distribución de carga eficiente entre los núcleos del procesador. Sin embargo, al alcanzar los límites físicos de los hilos, se observa un rendimiento decreciente.

Para validar el correcto comportamiento físico del modelo, se implementó la función `Paraview_data`, encargada de exportar periódicamente el estado de la distribución de calor a archivos `.vtk`. El procesamiento de estos archivos mediante ParaView permitió visualizar la evolución térmica de la simulación. Al analizar el estado final, se puede observar claramente la influencia de las condiciones de frontera impuestas sobre el sistema:



En esta primera visualización tridimensional, se puede evidenciar que la temperatura uniforme de 100.0 esta impuesta en casi la totalidad del perímetro exterior, contrastando con la única cara configurada en 0.0. La visualización demuestra que la aplicación inicial del estado de frontera se mantuvo íntegra a lo largo de las iteraciones.



La perspectiva en sección transversal muestra un decaimiento suave y progresivo de la temperatura, evidenciando una distribución de calor estacionaria correcta dictada por el *estencil* de 6 vecinos. La visualización confirma matemática y gráficamente que la simulación cumple con las reglas físicas establecidas en la descripción del problema, validando así la funcionalidad completa tanto de la solución secuencial como de la paralela.